

Programación de Redes I – Tarea 2

Pedro D. Llerenas

pedro.llerenas@cimat.mx

2026-02-25

1. Dada una imagen de entrada, obtenga la frecuencia de cada uno de sus pixeles usando 1, 2, 4 procesos.
1. Determine el tiempo de ejecución para cada caso (ver ANEXO 1);
2. ¿El tiempo de ejecución con 2 y 4 procesos es 2 y 4 veces menor que un solo proceso?

1. Para compilar y ejecutar el código, usamos

```
gcc fork_img_freq.c pgm_io.c -o fork_freq
./fork_freq images/lena_ascii.pgm 1
./fork_freq images/barbara_ascii.pgm 1
./fork_freq images/fractal_ascii.pgm 1
```

Donde el primer argumento es la imagen, y el segundo el número de procesos a usar. La siguiente tabla contiene los tiempos de CPU según el número de procesos e imágenes usadas.

N	Lena	Barbara	Fractal
1	0.000339s	0.000311s	0.003149s
2	0.000392s	0.000386s	0.003490s
4	0.000572s	0.000601s	0.003756s

Tabla 1: Comparación de tiempos de cálculo de frecuencias en imagen.

2. No. Parece incrementar, que puede ser debido al hecho de que al final de cada proceso hijo, el padre debe realizar la suma de cada elemento del arreglo del proceso hijo, haciendo que los otros hijos esperen.

2. Implementar la suma y multiplicación de matrices 3D mediante J procesos creados con la función **fork()**. Usar matrices cuadradas cuyas dimensiones y número de procesos son dados como línea de comando y presentar una tabla comparativa de los tiempos de ejecución.

Para compilar los programas, usamos

```
gcc matrix_product.c -o matrix_p
gcc matrix_sum.c -o matrix_s
```

Para ejecutar el producto de 2 matrices, usamos

```
./matrix_p <Dimension> <Procesos> <Semilla>
```

Por ejemplo:

```
./matrix_p 1000 1 101
```

Para la suma, usamos

```
./matrix_s <Dimension_1> <Dimension_2> <Dimension_3> <Procesos> <Semilla>
```

Por ejemplo,

```
./matrix_s 300 300 300 1 101
```

La siguiente tabla muestra algunos resultados obtenidos:

N	M	K	P = 1	P = 2	P = 4
100	100	100	0.000278s	0.000688s	0.000824s
300	300	300	0.001560s	0.002993s	0.006633s
500	500	500	0.005981s	0.011787s	0.023188s

Tabla 2: Comparación de tiempos para cálculo de suma de matrices.

N	P = 1	P = 2	P = 4
100	0.000127s	0.000215s	0.000363s
300	0.000144s	0.000220s	0.000456s
500	0.000173s	0.000347s	0.000541s
1000	0.000218s	0.000359s	0.000666s

Tabla 3: Comparación de tiempos para cálculo de producto de matrices.

Por lo observado, usar fork añade un overhead significativo, lo que hace que el rendimiento baje a comparación de un solo proceso.

3. Dada una imagen de entrada, calcular el filtro de la media y mediana usando $N = \{1, 2, 4, 9\}$ procesos. Presentar una tabla comparativa con los tiempos de ejecución.

Para compilar el código, usamos

```
gcc filter.c kernel_io.c pgm_io.c convolution.c compare.c -o filter
```

Para ejecutarlo, usamos

```
./filter <input> <output> <filter> <window size> <processes>
```

Por ejemplo:

```
./filter images/lena_ascii.pgm images/lena_mean.pgm mean 3 1
```

Las siguientes tablas muestran los tiempos de ejecución

N	Lena	Barbara	Fractal
1	0.000600s	0.000668s	0.005762s
2	0.000807s	0.000674s	0.005940s
4	0.000817s	0.000829s	0.006473s
9	0.001381s	0.001268s	0.007644s

Tabla 4: Comparativa de tiempos de ejecución para filtro de la media.

N	Lena	Barbara	Fractal
1	0.000444s	0.000409s	0.003214s
2	0.000722s	0.000521s	0.003064s
4	0.000765s	0.000696s	0.003526s
9	0.001213s	0.001129s	0.003897s

Tabla 5: Comparativa de tiempos de ejecución para filtro de la mediana.